



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Lab Report On

Process Scheduling: Calculation of Waiting Time and Average Waiting Time

Course: Operating System and System Programming Lab

Course Code: CSE 3102

Submission Date: 25/06/2025

Submitted by	Submitted to
Name: Adnan Zaman Niloy Student ID: 210142 Year: 3rd Semester: 1 st Session: 2021-22	Monishanker Halder, Assistant Professor, Department of Computer Science and Engineering, Jashore University of Science and Technology

Experiment Name:

Process Scheduling: Calculation of Waiting Time and Average Waiting Time

Introduction:

Process scheduling is a key component of modern operating systems. It determines the order in which processes access the CPU. Among various scheduling algorithms, calculating **waiting time** and **average waiting time** plays a crucial role in evaluating scheduling efficiency.

In this experiment, we simulate two basic CPU scheduling scenarios:

- **Problem 01:** Processes with burst time only (FCFS without arrival time)
- **Problem 02:** Processes with both arrival time and burst time (FCFS with arrival time)

Methodology:

Problem 01: Without Arrival Time (FCFS)

- **Input:** Number of processes and their burst times
- **Assumption:** All processes arrive at time 0
- **Waiting Time Calculation:**
 - First process waiting time = 0
 - Each subsequent process waits for the sum of burst times of all previous processes

Code Snippet:

```
for (int i = 1; i < n; i++)  
    waiting[i] = waiting[i - 1] + burst[i - 1];
```

Problem 02: With Arrival Time (FCFS)

- **Input:** Process name, arrival time, and burst time
- **Method:**
 - Sort processes based on arrival time
 - Track current CPU time
 - If the next process hasn't arrived yet, the CPU stays idle
 - Waiting time = current CPU time - arrival time

Sorting Code Snippet:

```
sort(p.begin(), p.end(), [](const Process &a, const Process &b) {  
    return a.arrival < b.arrival; });
```

Waiting Time Calculation:

```
for (int i = 0; i < n; ++i) {  
    if (current_time < p[i].arrival)  
        current_time = p[i].arrival;  
  
    p[i].waiting = current_time - p[i].arrival;  
    total_waiting += p[i].waiting;  
  
    cout << p[i].name << "\t" << p[i].arrival << "\t" << p[i].burst << "\t" << p[i].waiting  
    << "\n";  
    current_time += p[i].burst;  
}
```

Results & Discussion:

Problem 01:

Input:

5

1 2 3 4 5

Output

Process 1: Waiting time = 0

Process 2: Waiting time = 1

Process 3: Waiting time = 3

Process 4: Waiting time = 6

Process 5: Waiting time = 10

Average waiting time: 4.00

Grant chart:

0	1	3	4	10
P1	P2	P3	P4	P5

Problem 02:

Input: 4 p1 4

10 p2 5

5 p3 8

7 p4 4

Output

Process	Arrival	Burst	Waiting
p1	4	10	0
p4	4	0	10
p2	5	5	9
p3	8	7	11

Average waiting time: 7.50

Grant Chart:

0	14	14	19
26			
P1	P4	P2	P3

Discussion:

- **Problem 01:**
FCFS without arrival time executes processes in input order, resulting in a moderate **average waiting time of 4.00 units.**

- **Problem 02:**

Due to varying arrival times, the CPU remains idle at times, and some processes wait longer. This results in a **higher average waiting time of 12.00 units**, demonstrating how FCFS performs poorly with non-zero arrival times.

Conclusion:

This experiment demonstrates how CPU scheduling policies impact process execution.

- **FCFS is simple but not always efficient**, especially when arrival times vary.
- Waiting time and average waiting time are key metrics to evaluate performance.
- In real-world systems, more advanced algorithms (like SJF, RR, or Priority Scheduling) may be preferred to improve CPU utilization and minimize waiting time.